Statement of Qualifications
Uriah Raffer | JC-522217 | Information Technology Associate

1. Explain which AWS security tools or services you have worked with. How did you use them to protect cloud resources and what impact did your work have?

These come from personal Amazon Web Services (AWS) lab projects I built and documented myself. The infrastructure itself, a two-availability-zone environment with auto-scaling groups and a Multi-AZ database, was built for high availability. The security came from the controls I put on top of it. I use Identity and Access Management (IAM) to define roles with least-privilege policies so each identity only gets the access its job needs, and I use AWS Systems Manager Session Manager (SSM) to reach the instances instead of Secure Shell (SSH), which keeps that access gated by IAM and recorded. I also kept the network tight, splitting each environment into public and private subnets with security groups so my private hosts have no open inbound ports and no public addresses.

Beyond access, I send account and API activity to CloudTrail and set up CloudWatch metric filters, alarms, and an operations dashboard for visibility. I got the most hands-on with these while stress-testing the environment. I used a Bash script that terminated instances in every failure combination and recorded the events and timings, then traced the exact sequence in the CloudTrail event history, like an instance terminating, its replacement launching, and a Python Lambda repairing the route. One full run of that script produced 92 CloudTrail events, including 15 route replacements with zero false positives. Reading and correlating that trail is the same skill I would use to reconstruct who did what during a security incident. I also encrypt my database storage at rest with AWS Key Management Service (KMS) and enforce Instance Metadata Service v2 (IMDSv2) so the older, unauthenticated version can't be abused to pull an instance's credentials.

All of this is defined in Terraform, which matters for security and not just convenience. The configuration is version-controlled and reproducible instead of clicked together by hand, so it comes back identical every time I rebuild. It also ended a real headache, since building and tearing things down manually meant fighting dependency errors where one resource had to be removed before another in the right order, and Terraform handles that whole sequence with a single command. Layered this way, no single control failing would leave the environment exposed.

2. Describe a situation where you assisted in resolving a security-related issue in AWS. What was your approach and what did you learn from the experience?

This came out of my own AWS lab. Early on I had one thing right by following best practice and keeping the account root user out of my daily work. I created an IAM user with administrator access for hands-on tasks and left root reserved for the few account-level actions that actually require it. The weakness I caught was elsewhere. I was pointing my Terraform automation at that same broad administrator identity and driving it with long-lived credentials, and I was reaching my instances over SSH with key pairs and an open port. My automation was running with far more power than it needed, on credentials that never expired.

My approach was to harden it on two fronts. First, I removed SSH entirely and switched to SSM, which eliminated SSH as an attack surface. There is no open port to reach, no password to brute force, and no key to steal or reuse, because a session can only be opened by an authorized user through SSM. Second, I stopped running automation under that admin account. I created a dedicated role scoped to only what Terraform needed, backed by AWS Single Sign-On (SSO) so it issues temporary credentials instead of long-lived keys. That gave each identity one defined purpose, instead of one account that could do everything. Before I trusted the lockdown, I confirmed I could still reach every instance through SSM with SSH fully closed, so the change didn't cost me access.

The payoff was a much smaller exposure. If any one credential leaked now, it would reach a single narrow job rather than my whole account. What I took from it is that least privilege and separation of duties matter most for the identities you aren't watching in real time. I now plan those identities in from the start and save the powerful credentials for the rare cases that genuinely need them.